Smart Contract Assessment

0x Project

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

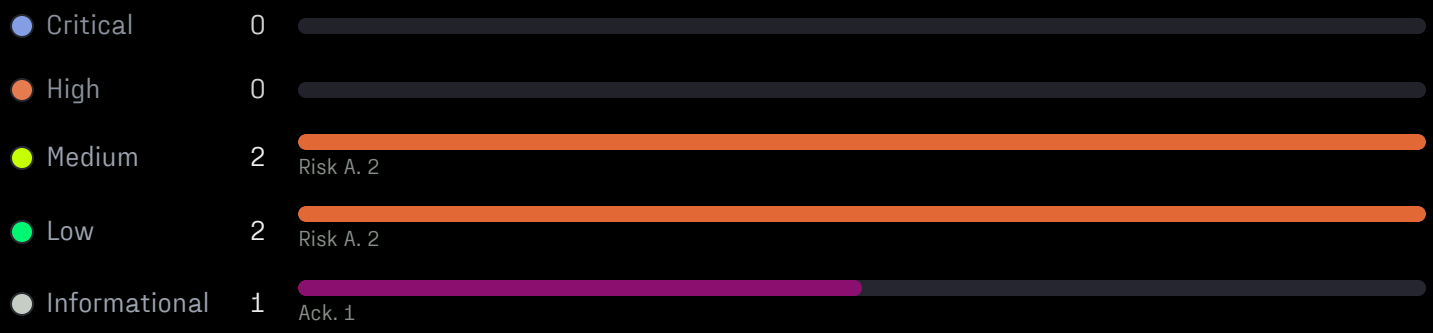
DATE OF ENGAGEMENT
Sep 29, 2026 - Oct 1, 2026

SOURCE CODE
Ox-settler

ASSESSED COMMIT ID
f358d81

SEVERITY BREAKDOWN

5	4	1	0
All Findings	Risk Accepted	Acknowledged	Solved



INTRODUCTION

`0x` team engaged `Halborn` to conduct a security assessment on their smart contracts beginning on September 30th, 2026 and ending on October 1st, 2026. The assessment scope was limited to the smart contracts provided to `Halborn`. Commit hashes and additional details are available in the Scope section of this report.

The reviewed contracts are part of OxSettler, a DEX aggregator settlement system that executes each swap as an ordered list of actions, pulling the sell token through Permit2 or AllowanceHolder and enforcing a slippage check on the output. The review covered `Select`, which adds a route selector that runs each candidate route in a revertible self call and commits the first one to meet its output target; `SettlerBase`, which decodes and routes actions and implements the final slippage check, together with the mainnet dispatch mixin `MainnetMixin`; and the `Permit2Payment` contracts, which manage the transient payer, operator and witness state and the token transfer paths of the taker submitted, metatransaction and intent flavors. The repository's `audits` directory contains prior audit reports, and the issues acknowledged in them are treated as known and are not reported again here.

ASSESSMENT SUMMARY

`Halborn` was provided with 2 days for this engagement and assigned 1 full-time security engineer to assess the security of the smart contracts in scope. The assigned engineer possesses deep expertise in blockchain and smart contract security, including hands-on experience with multiple blockchain protocols.

The objective of this assessment is to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, `Halborn` identified several areas for improvement to reduce the likelihood and impact of security risks, which were accepted by the `0x` team. The main recommendations were:

- `Approve caller-supplied spenders only for the exact amount sold and restrict the ERC-1820 registry as a target to stop later transactions from reaching tokens Settler holds mid-swap.`
- `Cap the proportional sell amount at a signed maximum to keep deposits that arrive after signing from being sold at the original floor.`
- `Check the estimated amount against the pool's own price and reduce it until it fits within the sell amount to prevent the pool from rejecting the trade and reverting the whole swap.`

TEST APPROACH AND METHODOLOGY

`Halborn` performed a combination of manual review of the code and automated security testing to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of the smart contract assessment. While manual testing is recommended to uncover flaws in logic, process, and implementation; automated testing techniques help enhance coverage of smart contracts and can quickly identify items that do not follow security best practices.

The following phases and associated tools were used throughout the term of the assessment:

- Research into the architecture and purpose of the in-scope contracts.
- Manual code review and walkthrough of the in-scope contracts.
- Manual assessment of critical `Solidity` variables and functions to identify potential vulnerability classes.

- Manual testing using custom scripts.
- Static Analysis of security for scoped contract, and imported functions (`Slither`).
- Local deployment and testing with (`Foundry`, `Remix IDE`).

Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational

9 - 10	7 - 8.9	4.5 - 6.9	2 - 4.4	0 - 1.9

Scope

REPOSITORY

(a) Repository: Ox-settler

(b) Assessed Commit ID: f358d811483fdad25486b34b5ca20e4d4393eaf6

(c) Items in scope:

- src 4 files
  - chains/Mainnet 1 file
    - Common.sol Solidity
  - core 2 files
    - Permit2Payment.sol Solidity
    - Select.sol Solidity
  - SettlerBase.sol Solidity

Out-of-Scope: Third party dependencies and economic attacks.

Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	Attacker can skim other users' in flight tokens through a permanent allowance planted with BASIC	Medium	5.6	RISK ACCEPTED 10/19/2026
HAL-002	Solver can keep the full value of bridge wallet deposits that arrive after the intent is signed	Medium	5.0	RISK ACCEPTED 10/19/2026
HAL-003	Recipients of fee on transfer buy tokens can receive less than the minimum output	Low	2.5	RISK ACCEPTED 10/19/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-004	DODOV1 quote to base sales revert whenever the local price estimate exceeds the pool's actual cost	Low	2.5	RISK ACCEPTED 10/19/2026
HAL-005	Excess ETH is left unprotected when NATIVE_CHECK runs inside a SELECT candidate	Informational	1.7	ACKNOWLEDGED 10/19/2026

Findings & Tech Details

HAL-001 RISK ACCEPTED

Attacker can skim other users' in flight tokens through a permanent allowance planted with BASIC

Medium · 5.6 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:M/Y:N

DESCRIPTION

`MainnetMixin._dispatch()` forwards the `BASIC` and `DODOV1` actions to `Basic.basicSellToPool()` and `DodoV1.sellToDodoV1()`, which approve the caller chosen `pool` or `dodo` address through `SafeTransferLib.safeApproveIfBelow()`. That helper always raises the allowance to `type(uint256).max` and nothing ever revokes it, while `_isRestrictedTarget()` only excludes Permit2, AllowanceHolder and Bebop, which means any caller can run a one wei `BASIC` sale against a contract they deployed and leave the `MainnetSettler` contract with a permanent unlimited allowance to that contract for the chosen token.

The allowance becomes harmful in later swaps of other users, because Settler holds their tokens between the actions of a swap. Once a victim's route invokes code the attacker controls, such as a hook attached to one of the pools the route trades through, which the `UNISWAPV4` action accepts from any address, that code can call `transferFrom` on Settler and take the victim's tokens that are still waiting for a later hop. A victim who sells 100 units of a token split evenly across an honest hooked pool and a second pool, with a 1% tolerance, receives 99 units of the buy token instead of 100 while the hook keeps 1 unit of the sell token, and without the planted allowance the same route delivers the full 100. The loss is capped per swap by the victim's `minAmountOut`, since taking more makes the final check revert, yet it repeats for every swap that routes through the attacker's code, and it contradicts the README statement that Settler has no outstanding allowances.

The same arbitrary call primitive extends the reach of this pattern past standing allowances. Because `basicSellToPool()` rejects only Permit2, AllowanceHolder and Bebop and otherwise forwards attacker chosen calldata to any address, and because the `address(0)` sell token branch runs that call with no amount or approval, an attacker can make Settler call the canonical ERC-1820 registry and register an `ERC777TokensSender` implementer for itself, which the registry allows without acknowledgement since Settler is its own default manager. That hook then fires on every later swap in which Settler transfers an ERC-777 token, giving the attacker a callback with no need to appear in the route, and a hook that reverts blocks those swaps outright. This path is limited to ERC-777 tokens, which are rare on mainnet.

SOL 105-116

```
// MainnetMixin._dispatch()
  } else if (action == uint32(ISettlerActions.BASIC.selector)) {
      (IERC20 sellToken, uint256 ppm, address pool, uint256 offset, bytes memory _data) =
          abi.decode(data, (IERC20, uint256, address, uint256, bytes));

      basicSellToPool(sellToken, ppm, pool, offset, _data);
      //...
  } else if (action == uint32(ISettlerActions.DODOV1.selector)) {
      (IERC20 sellToken, uint256 ppm, IDodoV1 dodo, bool quoteForBase, uint256
minBuyAmount) =
          abi.decode(data, (IERC20, uint256, IDodoV1, bool, uint256));

      sellToDodoV1(sellToken, ppm, dodo, quoteForBase, minBuyAmount);
```

SOL 21-29

```
// Basic.basicSellToPool()
function basicSellToPool(IERC20 sellToken, uint256 ppm, address pool, uint256 offset,
bytes memory data) internal {
  if (_isRestrictedTarget(pool)) {
  //...
      if (address(sellToken) != pool) {
          sellToken.safeApproveIfBelow(pool, amount);
      }
  }
  (success, returnData) = payable(pool).call{value: value}(data);
```

SOL 360-361

```
// DodoV1.sellToDodoV1()
  sellToken.safeApproveIfBelow(address(dodo), sellAmount);
```

RECOMMENDATION

Approve the exact amount being sold instead of the maximum, so that the allowance returns to zero once the spender pulls it. Add an exact approval helper to `SafeTransferLib` and use it in `Basic.basicSellToPool()` and `DodoV1.sellToDodoV1()`.

REMEDIATION COMMENT

RISK ACCEPTED: The `0x` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> Any malicious hook or pool on a route can already cost the taker everything above their `minAmountOut`, so the allowance adds nothing new. The ERC-1820 part is known — `BASIC` can

> make Settler register an ERC-777 sender hook. ERC-777 is rare, so we're not changing Settler

---

HAL-002 RISK ACCEPTED

Solver can keep the full value of bridge wallet deposits that arrive after the intent is signed

Medium · 5.0 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:H/Y:N

DESCRIPTION

When the payer is a proxy wallet deployed by the `CrossChainReceiverFactory` contract, `Permit2PaymentIntent._toCanonicalSellAmount()` reads a `permitted.amount` above `type(uint256).max - 1_000_000` as a proportion of the wallet's live token balance, which is resolved only at the moment the solver fills the order. The only price protection the wallet signs for an intent is the `Slippage` witness, whose `minAmountOut` is an absolute amount chosen for the balance expected at signing time, and nothing ties that floor to the amount that is actually sold.

Any deposit that reaches the wallet between signing and filling is therefore sold against the original floor, and since the whitelisted solver authors every action of an intent, it can route everything above `minAmountOut` to itself with a `POSITIVE_SLIPPAGE` action. A wallet that commits to selling 100% of its balance for at least 30,000 USDC, expecting a 10 WETH deposit, and then receives a second 10 WETH deposit before the fill has all 20 WETH sold, worth 60,000 USDC, while the recipient receives 30,000 USDC and the solver keeps the other 30,000. With an absolute `permitted.amount` of 10 WETH the second deposit stays in the wallet, which shows that the exposure of principal comes from the proportional encoding rather than from the solver's normal discretion over price surplus.

SOL 679-690

```sol
// Permit2PaymentIntent._toCanonicalSellAmount()
function _toCanonicalSellAmount(IERC20 token, uint256 sellAmount) private view returns (uint256) {
    unchecked {
        if (~sellAmount < Constants.BASIS) {
            if (_msgSender().codehash == _BRIDGE_WALLET_CODEHASH) {
                sellAmount = Constants.BASIS - ~sellAmount;
                sellAmount = tmp().omul(token.fastBalanceOf(_msgSender()), sellAmount).unsafeDiv(Constants.BASIS);
            }
        }
    }
    return sellAmount;
}

```

SOL 250-259

```sol
// SettlerIntent.executeMetaTxn()
function executeMetaTxn(

```

```
253         AllowedSlippage memory slippage,
254         bytes[] calldata actions,
255         bytes32 /* zid & affiliate */,
256         address msgSender,
257         bytes calldata sig
    ) public virtual override onlySolver metaTx(msgSender, _hashSlippage(slippage)) returns
    (bool) {
258         return _executeMetaTxn(slippage, actions, sig);
259     }
```

RECOMMENDATION

Bind the proportional sell amount to a signed ceiling. Add a `maxSellAmount` field to the intent `Slippage` witness, update `SLIPPAGE_TYPE`, `SLIPPAGE_TYPEHASH`, `SettlerIntent._hashSlippage()` and `Permit2PaymentIntent._witnessTypeSuffix()` accordingly, keep the signed value in transient storage for the duration of `SettlerIntent.executeMetaTxn()`, and clamp the resolved amount in `Permit2PaymentIntent._toCanonicalSellAmount()`. Changing the witness type invalidates signatures and bridge wallet commitments made against the current type, so the change requires a coordinated migration.

REMEDIATION COMMENT

RISK ACCEPTED: The `0x` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> Only 0x's whitelisted / trusted solvers fill intents, and the intended use case for the the bridge wallet is single deposit only

HAL-003 RISK ACCEPTED

Recipients of fee on transfer buy tokens can receive less than the minimum output

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

DESCRIPTION

`SettlerBase._checkSlippageAndTransfer()` reads the Settler balance of the buy token, compares it with `minAmountOut` and only then transfers it to the recipient, which means the guarantee applies to what Settler holds rather than on what the recipient actually receives. For a buy token that charges a fee on every transfer, the final payout is itself taxed and the recipient ends up with less than the minimum that passed the check, and when the exact limit option of `CHECK_SLIPPAGE` is used the recipient receives less than the minimum on every such swap.

With a token that charges 1% per transfer, a swap that leaves 198 tokens in Settler passes a 198 token minimum and the recipient receives 196.02 tokens without any revert. The same check closes the metatransaction and intent flows, where the minimum output is the only output guarantee the user signs, while the protocol already supports such tokens on the input side through the `feeOnTransfer` flag of its flash accounting actions.

RECOMMENDATION

Measure the amount the recipient received and compare that against the minimum in `SettlerBase._checkSlippageAndTransfer()`. Buy tokens that charge a transfer fee then need the full balance payout instead of the exact limit option:

SOL 1-6

```sol
uint256 balanceBefore = buyToken.fastBalanceOf(recipient);
buyToken.safeTransfer(recipient, amountOut);
uint256 received = buyToken.fastBalanceOf(recipient) - balanceBefore;
if (received < minAmountOut) {
    revertTooMuchSlippage(buyToken, minAmountOut, received);
}

```

REMEDIATION COMMENT

RISK ACCEPTED: The `0x` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> The minimum bounds what Settler sends, so a recipient of a taxed token can receive less, including in MetaTxn and Intent. This is known and we're not adding a recipient balance check

---

HAL-004 RISK ACCEPTED

DODOV1 quote to base sales revert whenever the local price estimate exceeds the pool's actual cost

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

`MainnetMixin._dispatch()` routes the `DODOV1` action to `DodoV1.sellToDodoV1()`, which, when selling the quote token for the base token, computes the base amount to buy with `DodoSellHelper.dodoQuerySellQuoteToken()`, a local reimplementation of DODO V1 pricing, and then calls the pool's `buyBaseToken()` with the whole sell amount as `maxPayQuote`. The pool prices that purchase with its own independently rounded math and rejects it with `BUY_BASE_COST_TOO_MUCH` whenever the cost exceeds `maxPayQuote` by even one wei, which means any overshoot of the local estimate aborts the entire transaction.

Against DODO V1's own pricing, the estimate overshoots in about 4% of states with k between 0.0001 and 0.5, by up to 44 wei, and in about a third of states with k above 0.9, by up to 2.26 million wei, including pools at rest. On a pool with R equal to one, k of 0.5, a one to one price and 1,000 tokens of depth, selling 80 quote tokens through the `MainnetSettler` contract reverts while selling 120 fills and leaves 26 wei of quote token in Settler. No funds are lost, but a route that simulated cleanly can fail on chain once the pool state moves, which wastes the user's gas and fails signed metatransaction and intent orders.

SOL 192-197

```sol
// MainnetMixin._dispatch()
        } else if (action == uint32(ISettlerActions.DODOV1.selector)) {
            (IERC20 sellToken, uint256 ppm, IDodoV1 dodo, bool quoteForBase, uint256 minBuyAmount) =
                abi.decode(data, (IERC20, uint256, IDodoV1, bool, uint256));

            sellToDodoV1(sellToken, ppm, dodo, quoteForBase, minBuyAmount);

```

SOL 361-367

```sol
// DodoV1.sellToDodoV1()
        if (quoteForBase) {
            uint256 buyAmount = dodoQuerySellQuoteToken(dodo, sellAmount);
            if (buyAmount < minBuyAmount) {
                revertTooMuchSlippage(dodo.fast_BASE_TOKEN_(), minBuyAmount, buyAmount);
            }
            dodo.fastBuyBaseToken(buyAmount, sellAmount);

```

RECOMMENDATION

Check the estimate against the pool's own `queryBuyBaseToken()` and reduce it until the cost fits within the sell amount, with a bounded number of attempts. This requires adding `queryBuyBaseToken(uint256)` to the `IDodoV1` contract.

REMEDIATION COMMENT

RISK ACCEPTED: The `0x` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> A failed swap costs gas but moves no funds, so we're not adding a pool-cost check for now

---

HAL-005 ACKNOWLEDGED

Excess ETH is left unprotected when NATIVE_CHECK runs inside a SELECT candidate

• Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

DESCRIPTION

The `Select` contract runs every candidate through a self call that carries no value, and `Select._runActions()` dispatches each candidate action through `MainnetSettler._dispatch()` inside that frame. When a candidate contains `NATIVE_CHECK`, the `MainnetSettler._dispatch()` function compares the taker's bound against the `msg.value` of the trial frame, which is always zero, and the check therefore passes no matter how much ETH the taker actually attached to the transaction. The deadline half of the action is weakened in the same way, since an expired deadline only makes that one candidate fail and `Select.select()` moves on to the next candidate instead of stopping the swap.

The purpose of `NATIVE_CHECK` is to stop a taker from overpaying, since any ETH that remains in the `MainnetSettler` contract after the transaction can be taken by the next caller through a `POSITIVE_SLIPPAGE` action. A taker who sends 1 ETH with a candidate that contains `NATIVE_CHECK(deadline, 0)` completes the swap without a revert, and a third party then withdraws the full 1 ETH in a separate call. The same action placed at the top level of `actions` reverts with `MsgValueMismatch`, and the `SELECT` documentation in the `ISettlerActions` contract warns only against nesting `CHECK_SLIPPAGE`.

RECOMMENDATION

Reject context dependent actions inside trials in `Select._runActions()`, so that the candidate fails instead of silently passing.

REMEDIATION COMMENT

ACKNOWLEDGED: The `0x` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> The `SELECT` docs now say candidates must not contain `NATIVE_CHECK` or `CHECK_SLIPPAGE`, so value checks belong outside `SELECT`

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.